CLOUDSHIELD

DevSecOps Hackathon 2026

Securing the Future, One Commit at a Time

HACKATHON DOCUMENTATION

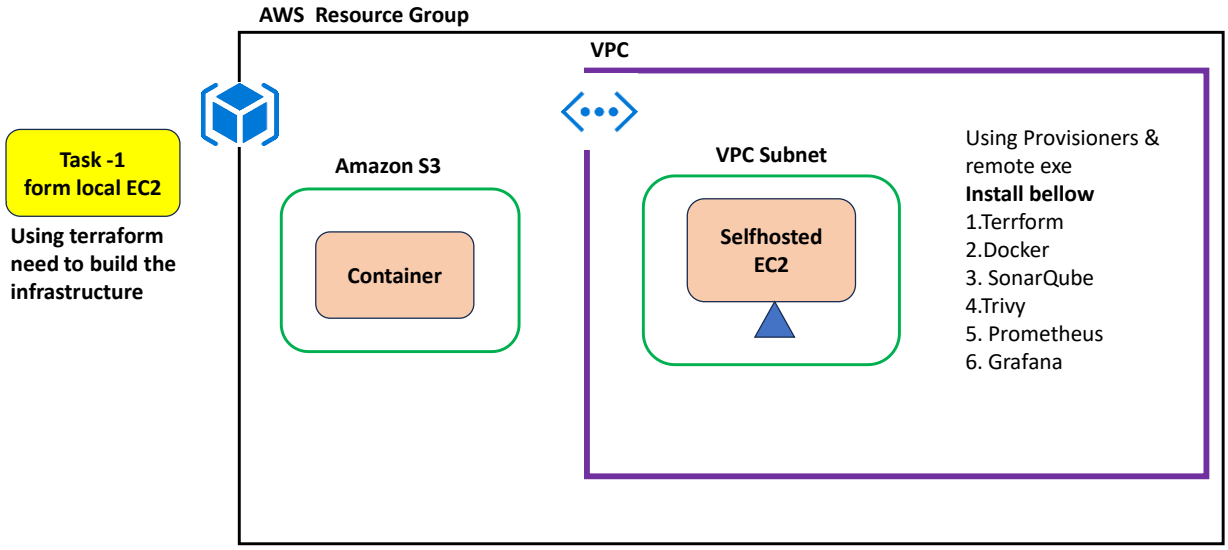
Documentation Intro

AWS Infrastructure Automation and CI/CD Pipeline

Tools Used:

- Terraform – Infrastructure as Code
- Amazon Web Services – Cloud infrastructure
- Docker – Containerization
- SonarQube – Code quality scanning
- Trivy – Container security scanning
- Prometheus – Monitoring
- Grafana – Visualization
- Kubernetes – Container orchestration
- Apache Maven – Java build tool

Task-1 Infrastructure Workflow (Terraform)



Terraform Infrastructure Flow

Terraform Apply

|

▼

Create VPC

|

▼

Create Subnet

|

▼

Create S3 Bucket

|

▼

Create Self-Hosted EC2

|

▼

Install Tools using Provisioners

|

▼

Terraform

Docker

SonarQube

Trivy

Prometheus

Grafana

Explanation:

Terraform creates:

Network Layer

- VPC
- Subnet

Storage

- S3 bucket (for containers or artifacts)

Compute

- EC2 instance (self-hosted DevOps server)

Provisioning

Terraform `remote-exec` installs:

- Docker
- SonarQube
- Trivy
- Prometheus
- Grafana

Terraform Infrastructure Deployment – Step by Step

Step 1: Configure AWS Provider

The provider configuration defines the cloud platform where the infrastructure will be deployed.

provider.tf

```
provider "aws" {

region = "ap-northeast-1"

backend "s3" {
bucket        = "devops-terraform-state-bucket"

key           = "devops-agent/terraform.tfstate"
region        = "ap-northeast-1"
```

```
  dynamodb_table = "terraform-lock-table"
  encrypt        = true
 }
}
```

Explanation

- Terraform connects to AWS.
- The region used for deployment is **Asia Pacific (Tokyo)**

Step 2: Create Virtual Private Cloud (VPC)

File: **vpc.tf**

```
resource "aws_vpc" "devops_vpc" {

cidr_block          = "192.0.0.0/16"

tags = {

Name = "DevOps-VPC"

}

}
```

Explanation

- This step creates a **Virtual Private Cloud** which acts as the private network for all infrastructure resources.

Step 3: Create Public Subnets

- Two public subnets are created in different availability zones for high availability.

Public Subnet 1

```
resource "aws_subnet" "public_subnet" {
```

```
  vpc_id                  = aws_vpc.devops_vpc.id
  cidr_block              = "192.0.7.0/24"
  availability_zone       = "ap-northeast-1a"
map_public_ip_on_launch = true
}
```

Public Subnet 2

```
    resource "aws_subnet" "public_subnet_2" {
  vpc_id                  = aws_vpc.devops_vpc.id
  cidr_block              = "192.0.8.0/24"
  availability_zone       = "ap-northeast-1c"
  map_public_ip_on_launch = true
}
```

Explanation

These subnets allow EC2 instances to access the internet.

Step 4: Create Internet Gateway

```
resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.devops_vpc.id
}
```

Explanation

The Internet Gateway allows resources in the VPC to communicate with the internet

Step 5: Configure Route Table

```
resource "aws_route_table" "public_rt" {
  vpc_id = aws_vpc.devops_vpc.id
route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }
}
```

Explanation

The route table directs internet traffic through the Internet Gateway

Step 6: Associate Subnets with Route Table

```
resource "aws_route_table_association" "rt_association_1" {
  subnet_id      = aws_subnet.public_subnet.id
  route_table_id = aws_route_table.public_rt.id
}
resource "aws_route_table_association" "rt_association_2" {
  subnet_id      = aws_subnet.public_subnet_2.id
  route_table_id = aws_route_table.public_rt.id
}
```

Explanation

This connects the subnets to the route table so they can access the internet.

Step 7: Create Security Group

File: **security.tf**

```
 resource "aws_security_group" "devops_sg" {
name   = "devops-sg"
vpc_id = aws_vpc.devops_vpc.id

ingress {
  description = "SSH"
  from_port   = 22
  to_port     = 22
  protocol    = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
}

ingress {
  description = "SonarQube"
  from_port   = 9000
  to_port     = 9000
  protocol    = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
}

ingress {
  description = "Prometheus"
  from_port   = 9090
  to_port     = 9090
  protocol    = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
}

ingress {
  description = "Grafana"
  from_port   = 3000
  to_port     = 3000
  protocol    = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
}

ingress {
  description = "Node Exporter"
  from_port   = 9100
  to_port     = 9100
  protocol    = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
}

egress {
  from_port   = 0
```

```
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

Explanation

This Terraform configuration creates a **Security Group** that controls inbound and outbound traffic for the DevOps EC2 instance.

Step 8: Create Provisioners & install.sh

```
#!/bin/bash

# Log everything for debugging
exec > /var/log/install-script.log 2>&1

sudo apt update -y

# Install Docker
sudo apt install docker.io -y
sudo systemctl enable docker
sudo systemctl start docker

# Install AWS CLI
sudo apt install unzip -y
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install

# Install dependencies
sudo apt install wget curl gnupg apt-transport-https software-properties-common -y

# Install Trivy
wget https://aquasecurity.github.io/trivy-repo/deb/public.key
sudo apt-key add public.key
echo "deb https://aquasecurity.github.io/trivy-repo/deb $(lsb_release -sc) main" |
sudo tee /etc/apt/sources.list.d/trivy.list
sudo apt update
sudo apt install trivy -y

# Install kubectl
curl -LO https://dl.k8s.io/release/v1.28.0/bin/linux/amd64/kubectl
chmod +x kubectl
sudo mv kubectl /usr/local/bin/

# Run SonarQube container
sleep 20
sudo docker run -d --name sonarqube -p 9000:9000 sonarqube:lts

# Create Prometheus user if not exists
id -u prometheus &>/dev/null || sudo useradd --no-create-home --shell /bin/false
prometheus

# Download Prometheus
wget https://github.com/prometheus/prometheus/releases/download/v2.47.0/prometheus-
2.47.0.linux-amd64.tar.gz
tar -xvf prometheus-2.47.0.linux-amd64.tar.gz

sudo cp prometheus-2.47.0.linux-amd64/prometheus /usr/local/bin/
sudo cp prometheus-2.47.0.linux-amd64/promtool /usr/local/bin/
```

```
sudo mkdir -p /etc/prometheus
sudo mkdir -p /var/lib/prometheus

# Prometheus configuration
cat <<EOF | sudo tee /etc/prometheus/prometheus.yml
global:
  scrape_interval: 15s

scrape_configs:

  - job_name: "prometheus"
    static_configs:
      - targets: ["localhost:9090"]

  - job_name: "node_exporter"
    static_configs:
      - targets: ["localhost:9100"]
EOF

# Permissions fix
sudo chown -R prometheus:prometheus /etc/prometheus
sudo chown -R prometheus:prometheus /var/lib/prometheus

# Prometheus service
cat <<EOF | sudo tee /etc/systemd/system/prometheus.service
[Unit]
Description=Prometheus
After=network.target

[Service]
User=prometheus
ExecStart=/usr/local/bin/prometheus \
--config.file=/etc/prometheus/prometheus.yml \
--storage.tsdb.path=/var/lib/prometheus

[Install]
WantedBy=default.target
EOF

sudo systemctl daemon-reexec
sudo systemctl daemon-reload
sudo systemctl enable prometheus
sudo systemctl start prometheus

# Install Node Exporter
wget
https://github.com/prometheus/node_exporter/releases/download/v1.6.1/node_exporter-
1.6.1.linux-amd64.tar.gz
tar xvf node_exporter-1.6.1.linux-amd64.tar.gz
sudo cp node_exporter-1.6.1.linux-amd64/node_exporter /usr/local/bin/

# Node Exporter service
cat <<EOF | sudo tee /etc/systemd/system/node_exporter.service
[Unit]
Description=Node Exporter
After=network.target

[Service]
User=prometheus
ExecStart=/usr/local/bin/node_exporter

[Install]
WantedBy=default.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable node_exporter
sudo systemctl start node_exporter

# Install Grafana
wget -q -O - https://packages.grafana.com/gpg.key | sudo apt-key add -
```

```
echo "deb https://packages.grafana.com/oss/deb stable main" | sudo tee
/etc/apt/sources.list.d/grafana.list

sudo apt update
sudo apt install grafana -y

sudo systemctl enable grafana-server
sudo systemctl start grafana-server

# Show running ports
sudo ss -tulnp | grep -E "3000|9000|9090|9100"
```

Step 9: Create EC2.tf

```
  resource "aws_instance" "devops_vm" {

  ami           = "ami-0f65fc8c24ec8d2a1"
  instance_type = "t3.large"
  key_name      = "Bastion"

  subnet_id              = aws_subnet.public_subnet.id
  vpc_security_group_ids = [aws_security_group.devops_sg.id]

  associate_public_ip_address = true

  tags = {
    Name = "DevOps-Agent"
  }

  # Upload script to EC2
  provisioner "file" {
    source      = "install.sh"
    destination = "/home/ubuntu/install.sh"

    connection {
      type        = "ssh"
      user        = "ubuntu"
      private_key = file("Bastion.pem")
      host        = self.public_ip
    }
  }

  # Execute the script
  provisioner "remote-exec" {
    inline = [
      "chmod +x /home/ubuntu/install.sh",
      "sudo /home/ubuntu/install.sh"
    ]

    connection {
      type        = "ssh"
      user        = "ubuntu"
      private_key = file("Bastion.pem")
      host        = self.public_ip
    }
  }

}
```

➢ Verification and Screenshots

This section verifies that the infrastructure created using Terraform is functioning correctly and that all DevOps tools were installed successfully on the EC2 instance.

Figure 1: EC2 Instance Created Using Terraform

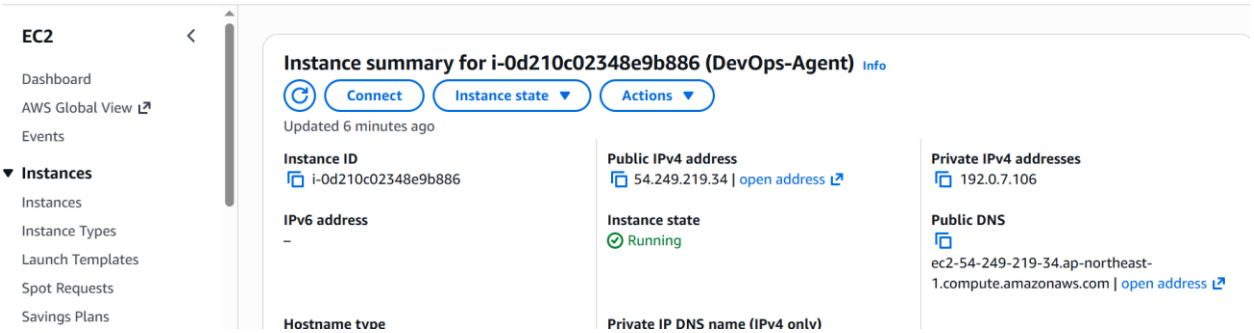


Figure 2: Security Group Configuration

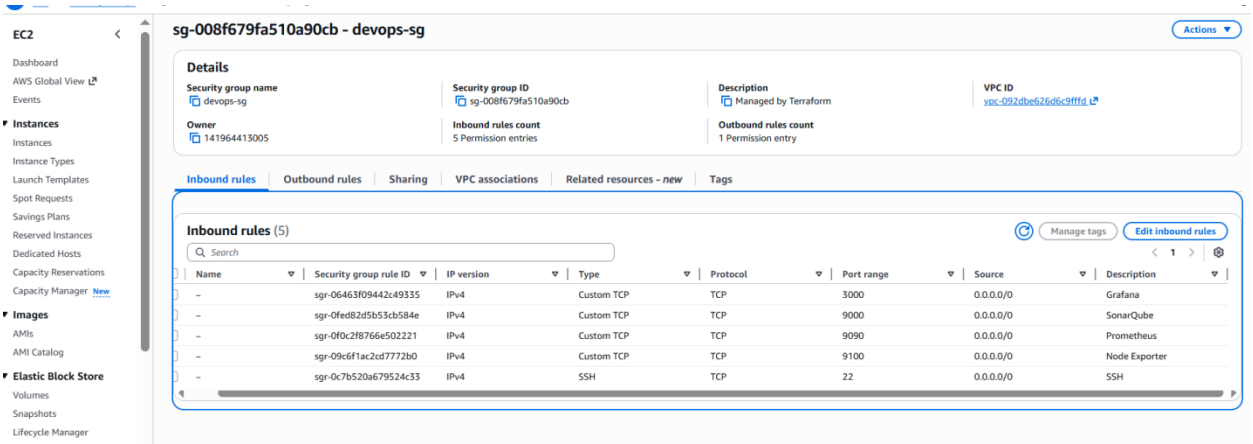


Figure 3: SonarQube Running on EC2 Instance

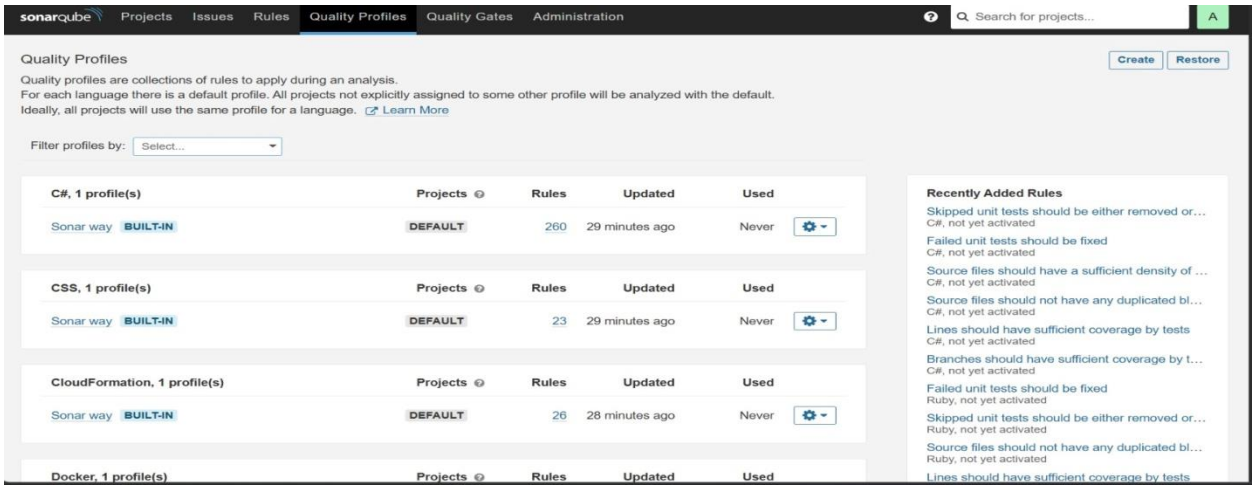


Figure 4: Prometheus Monitoring Targets

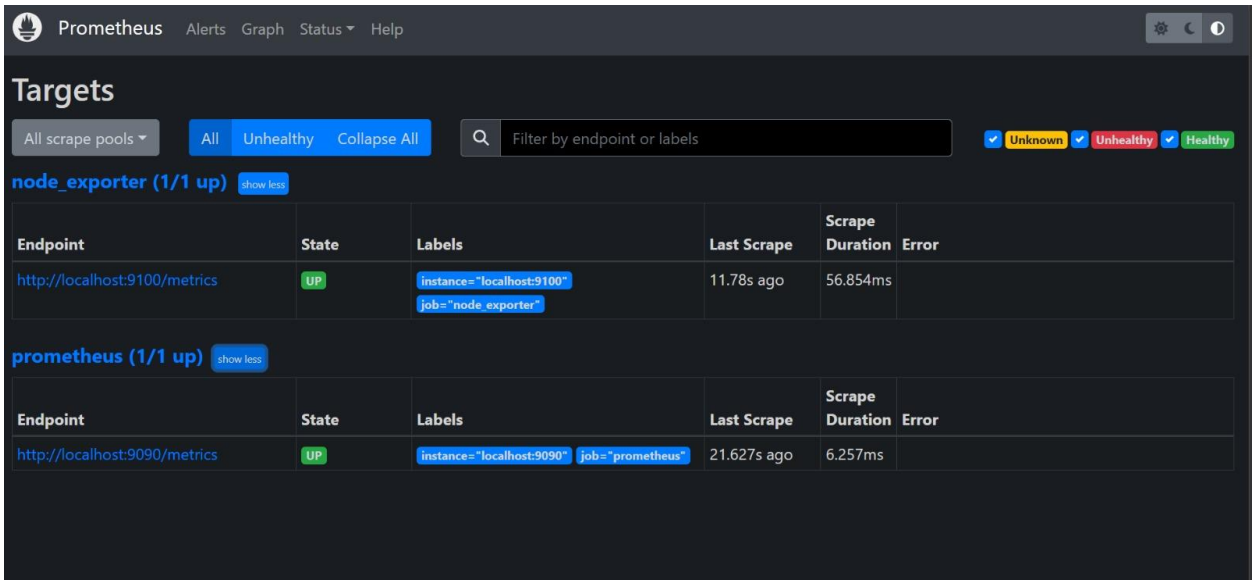


Figure 5: Grafana Monitoring Dashboard

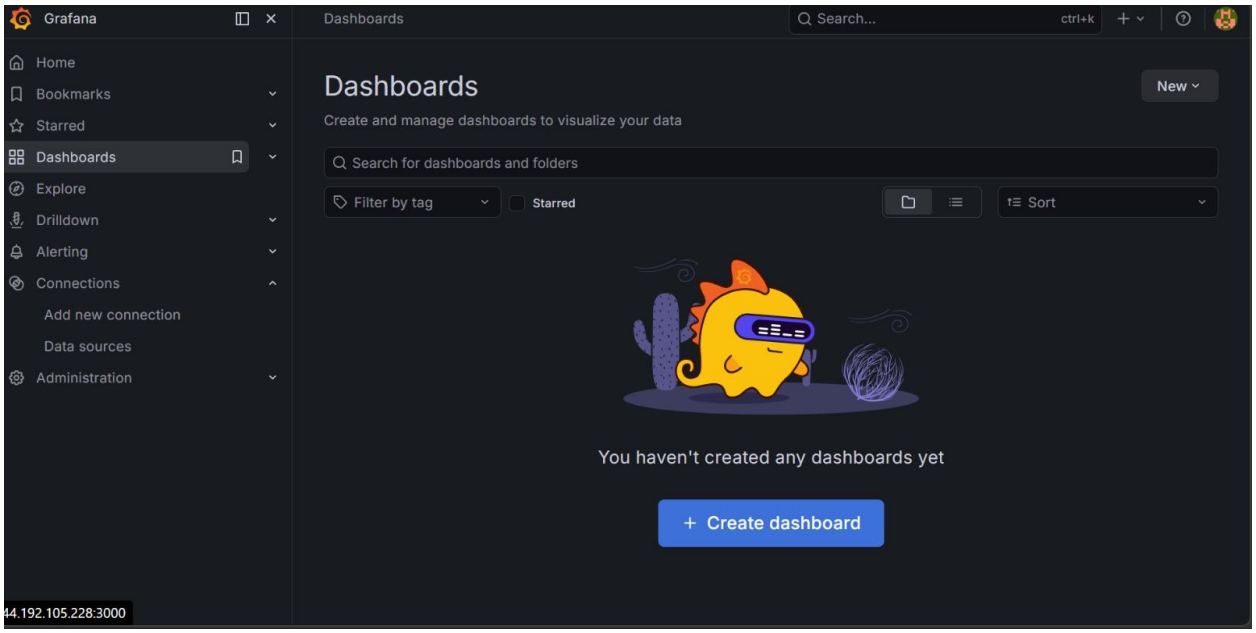
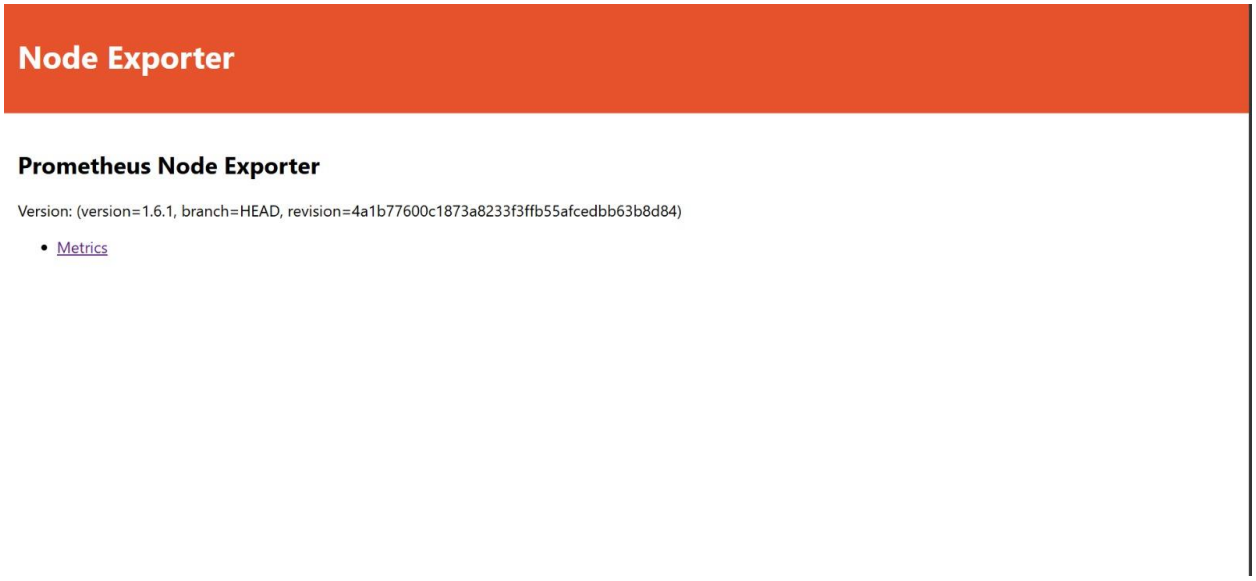
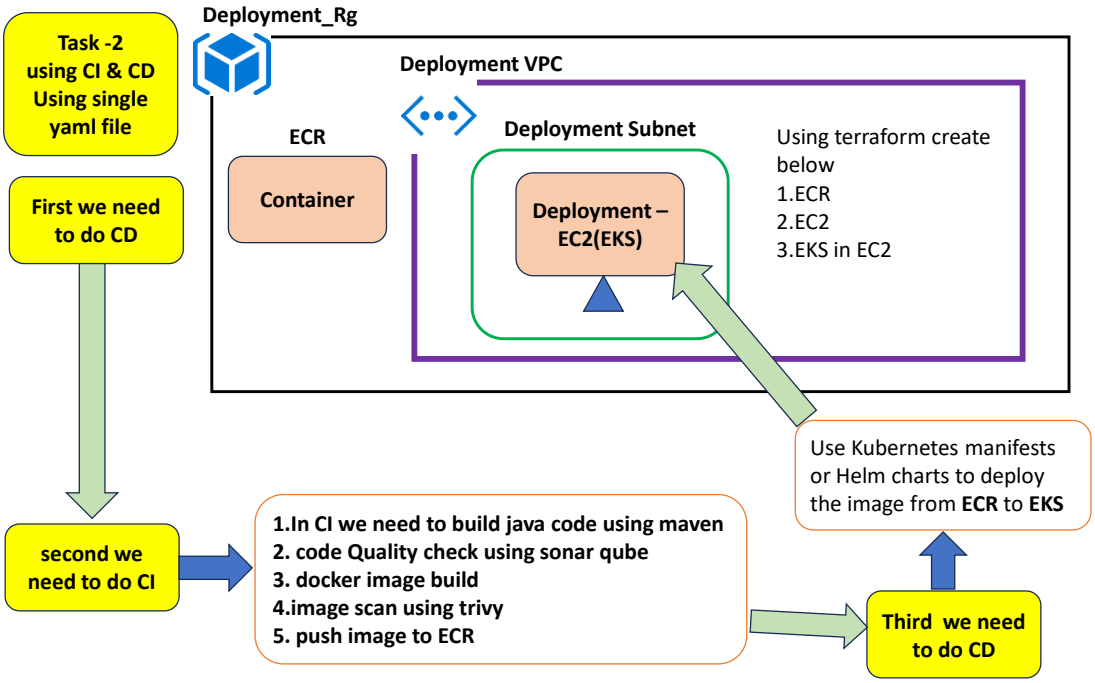


Figure 6: Node Exporter System Metrics



Task-2: CI/CD Pipeline Implementation Using YAML



1. Introduction

This task focuses on implementing a **CI/CD pipeline** that automates the build, scan, containerization, and deployment of a Java application.

The pipeline integrates multiple DevOps tools to ensure code quality, security scanning, and automated deployment to a Kubernetes environment.

Infrastructure resources such as the **container registry, compute instances, and networking components** are provisioned using **Terraform**.

The CI/CD workflow uses the following tools:

- Maven
- SonarQube
- Docker
- Trivy
- Kubernetes
- Helm
- Jenkins (or CI tool using YAML pipeline)

The application container image is stored in **Amazon Elastic Container Registry** and deployed to a Kubernetes cluster running on **Amazon Elastic Kubernetes Service** or EC2-based Kubernetes.

---

2. Architecture Overview

The CI/CD architecture consists of the following components:

1. **Terraform Infrastructure**
    - VPC
    - Subnet
    - EC2 DevOps Server
    - ECR Repository
2. **CI Pipeline**
    - Build Java application
    - Run code quality checks
    - Build Docker image
    - Scan image for vulnerabilities
    - Push image to ECR
3. **CD Pipeline**
    - Deploy application to Kubernetes using Helm or Kubernetes manifests

3. Infrastructure Provisioning

Infrastructure is provisioned using **Terraform** on AWS.

Terraform creates:

- VPC
- Subnets
- EC2 DevOps instance
- Security groups
- Amazon Container registry (ECR)
- Amazon Elastic Kubernetes Service(EKS)

The EC2 instance installs DevOps tools using a provisioning script.

Tools installed automatically:

- Docker
- SonarQube
- Prometheus
- Grafana
- Trivy
- Kubernetes CLI

3.1 Amazon ECR Repository Configuration

Amazon Elastic Container Registry is used to store Docker container images built during the CI pipeline.

Terraform Configuration

```
resource "aws_ecr_repository" "app" {

  name                 = var.ecr_repository_name

  image_tag_mutability = "MUTABLE"



  image_scanning_configuration {

    scan_on_push = true

  }



  tags = {

    Name = var.ecr_repository_name

  }

}



resource "aws_ecr_lifecycle_policy" "app" {

  repository = aws_ecr_repository.app.name



  policy = jsonencode({

    rules = [
```

```
    {
      rulePriority = 1
      description  = "Keep last 20 images"
      selection = {
        tagStatus   = "any"
        countType   = "imageCountMoreThan"
        countNumber = 20
      }
      action = {
        type = "expire"
      }
    }
  ]
 })
}
```

Explanation

- Creates an **ECR repository**
- Enables **automatic vulnerability scanning**
- Keeps **only the latest 20 images** using lifecycle policy

Figure 7: Amazon ECR Repository Created

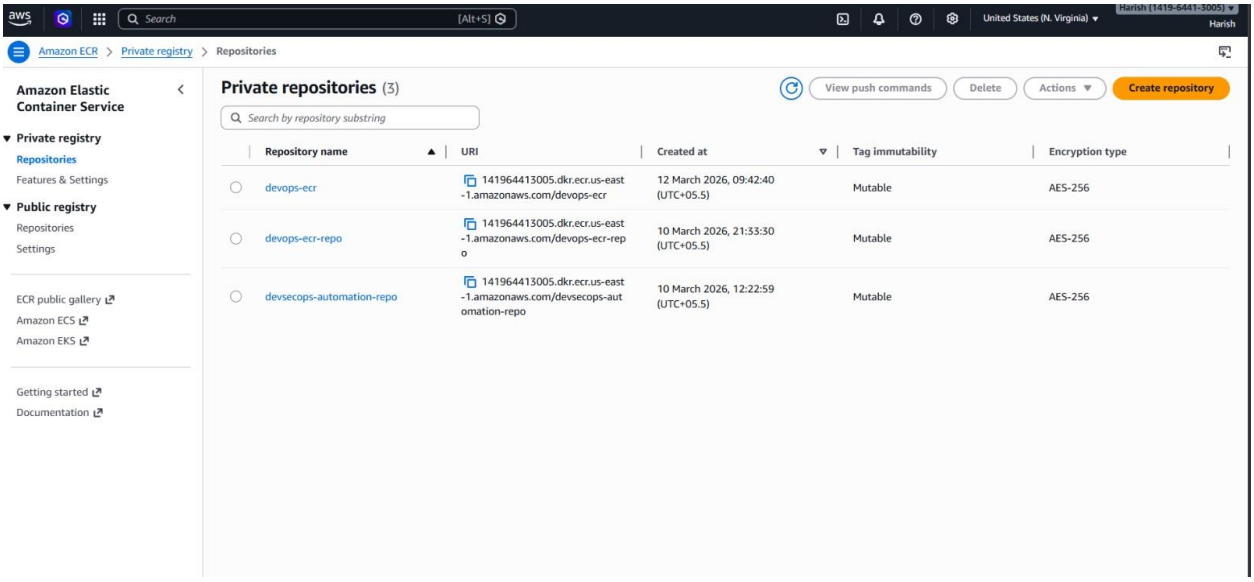


Figure 8: Docker Image Successfully Pushed to ECR

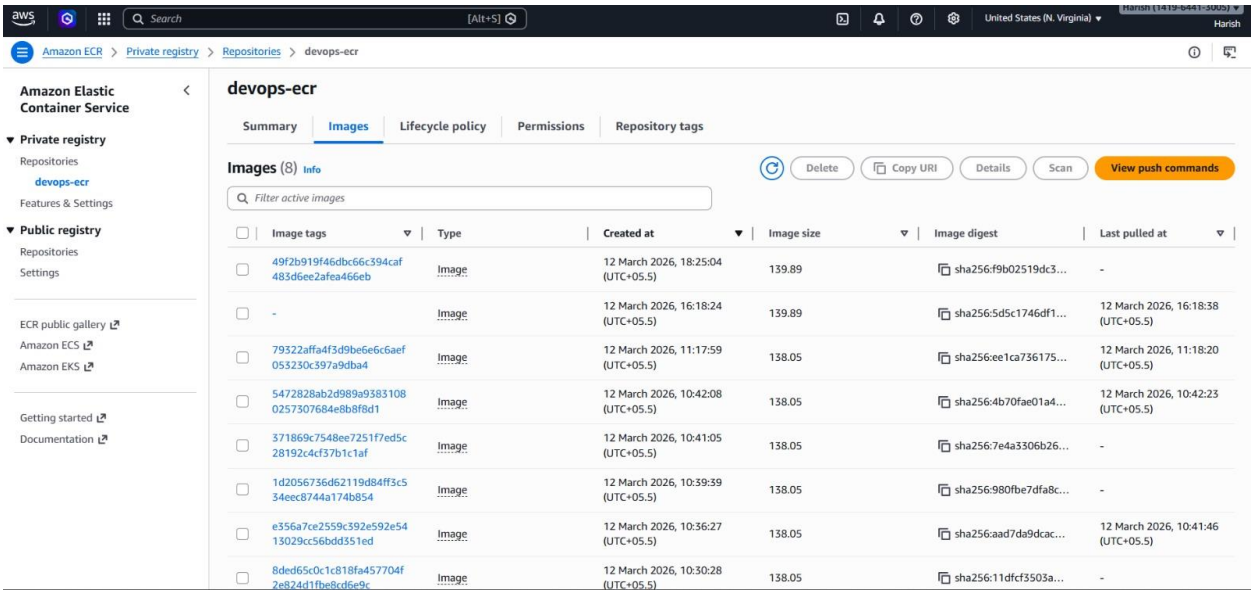
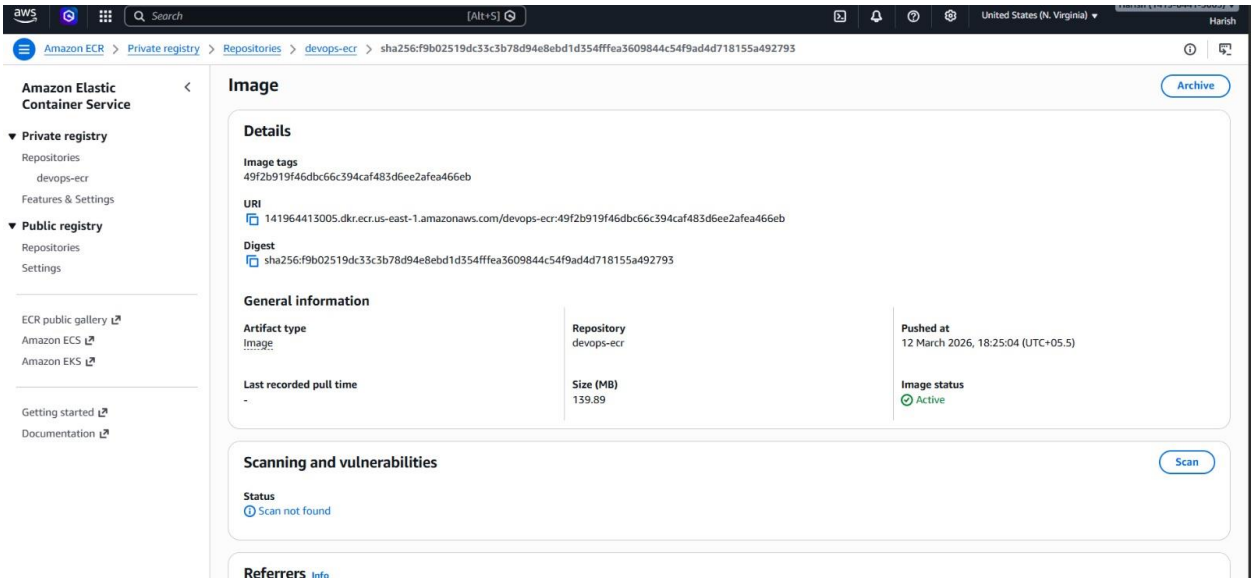


Figure 9: Docker Image Details ECR



3.2 Amazon EKS Cluster Configuration

```
############################
# EKS CLUSTER ROLE
############################

data "aws_iam_policy_document" "eks_cluster_assume_role" {
  statement {
    actions = ["sts:AssumeRole"]

    principals {
      type        = "Service"
      identifiers = ["eks.amazonaws.com"]
    }
  }
}

resource "aws_iam_role" "eks_cluster" {
  name               = "${var.eks_cluster_name}-cluster-role"
  assume_role_policy = data.aws_iam_policy_document.eks_cluster_assume_role.json
}

resource "aws_iam_role_policy_attachment" "eks_cluster_policy" {
  role       = aws_iam_role.eks_cluster.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKSClusterPolicy"
}

############################
# NODE ROLE
############################

data "aws_iam_policy_document" "eks_node_assume_role" {
  statement {
    actions = ["sts:AssumeRole"]

    principals {
      type        = "Service"
      identifiers = ["ec2.amazonaws.com"]
```

```
    }
  }
}

resource "aws_iam_role" "eks_node" {
  name               = "${var.eks_cluster_name}-node-role"
  assume_role_policy = data.aws_iam_policy_document.eks_node_assume_role.json
}

resource "aws_iam_role_policy_attachment" "eks_worker_node_policy" {
  role       = aws_iam_role.eks_node.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKSWorkerNodePolicy"
}

resource "aws_iam_role_policy_attachment" "eks_cni_policy" {
  role       = aws_iam_role.eks_node.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKS_CNI_Policy"
}

resource "aws_iam_role_policy_attachment" "eks_ecr_readonly" {
  role       = aws_iam_role.eks_node.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryReadOnly"
}

############################
# EKS CLUSTER
############################

resource "aws_eks_cluster" "main" {

  name     = var.eks_cluster_name
  role_arn = aws_iam_role.eks_cluster.arn

  access_config {
    authentication_mode                         = "API_AND_CONFIG_MAP"
    bootstrap_cluster_creator_admin_permissions = true
  }

  vpc_config {
    subnet_ids = [
      aws_subnet.public_subnet.id,
      aws_subnet.public_subnet_2.id
    ]
  }

  depends_on = [
    aws_iam_role_policy_attachment.eks_cluster_policy
  ]
}

############################
# NODE GROUP
############################

resource "aws_eks_node_group" "main" {

  cluster_name    = aws_eks_cluster.main.name
  node_group_name = "${var.eks_cluster_name}-nodegroup"
  node_role_arn   = aws_iam_role.eks_node.arn

  subnet_ids = [
    aws_subnet.public_subnet.id,
    aws_subnet.public_subnet_2.id
  ]

  instance_types = var.eks_node_instance_types

  scaling_config {
    desired_size = var.eks_node_desired_size
    min_size     = var.eks_node_min_size
    max_size     = var.eks_node_max_size
  }
```

```
  depends_on = [
    aws_iam_role_policy_attachment.eks_worker_node_policy,
    aws_iam_role_policy_attachment.eks_cni_policy,
    aws_iam_role_policy_attachment.eks_ecr_readonly
  ]
}
```

Explanation

- Creates **EKS Cluster IAM Role**
- Creates **Worker Node Role**
- Creates **EKS Cluster**
- Creates **Node Group for running containers**

Figure 10: Amazon EKS Cluster Created

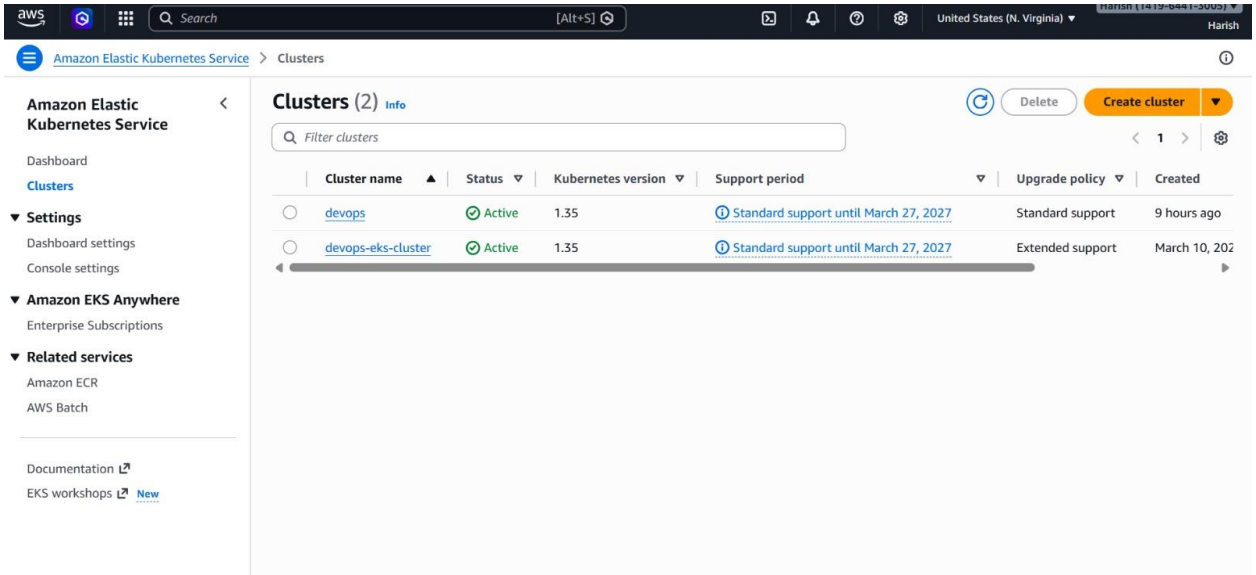


Figure 11: EKS Node Group Running

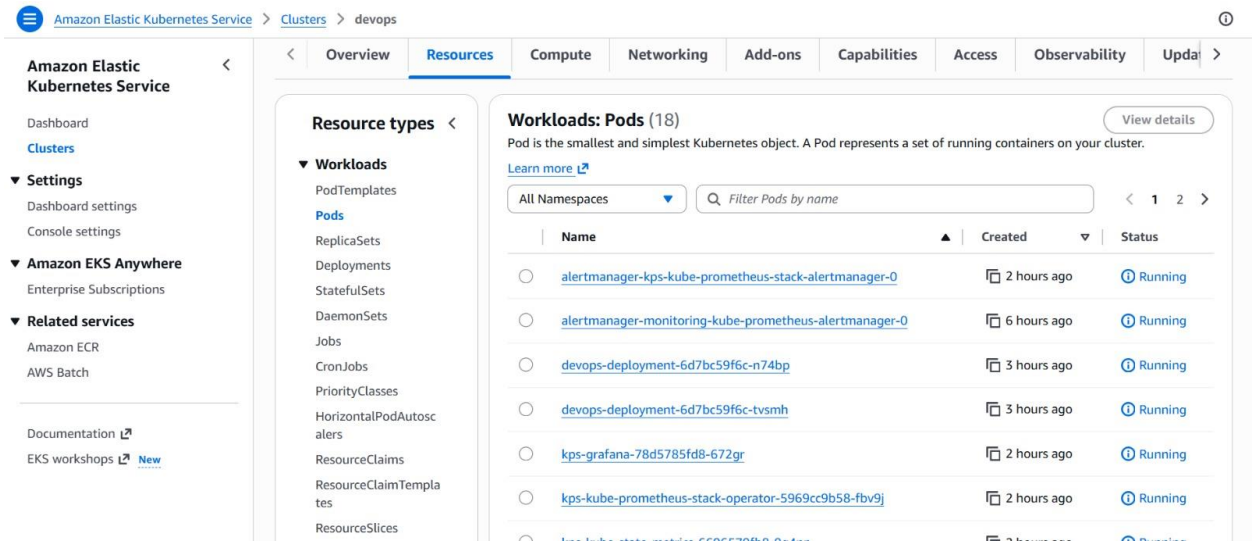


Figure 12: Worker Nodes Registered in the Cluster

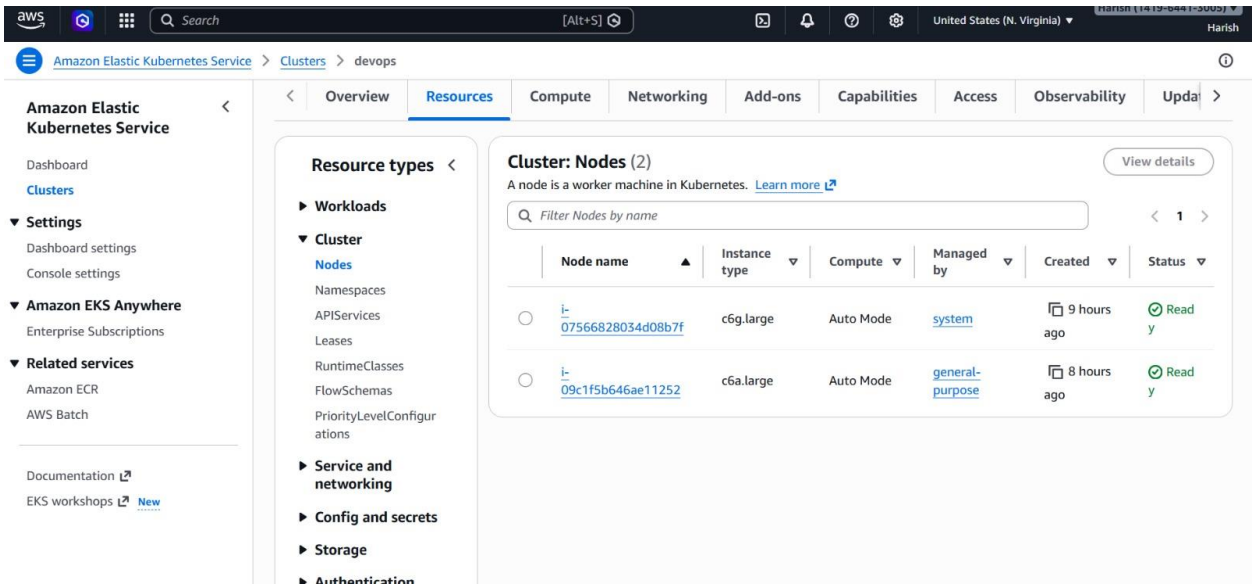


Figure 13: Kubernetes Pods Running

```
root@ip-10-0-1-87:~# docker --version
Docker version 28.2.2, build 28.2.2-0ubuntu1~22.04.1
root@ip-10-0-1-87:~# helm version
version.BuildInfo{Version:"v3.20.0", GitCommit:"b2e4314fa0f229a1de7b4c981273f61d69ee5a59", GitTreeState:"clean", GoVersion:"go1.25.6"}
root@ip-10-0-1-87:~# kubectl get po
NAME                                  READY   STATUS    RESTARTS   AGE
devops-deployment-6d7bc59f6c-n74bp    1/1     Running   0          158m
devops-deployment-6d7bc59f6c-tvsmh    1/1     Running   0          158m
root@ip-10-0-1-87:~# kubectl get svc
NAME             TYPE           CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
devops-service   LoadBalancer   10.100.172.92   <pending>     80:31181/TCP   8h
kubernetes       ClusterIP      10.100.0.1      <none>        443/TCP        8h
root@ip-10-0-1-87:~# kubectl get deployment
NAME               READY   UP-TO-DATE   AVAILABLE   AGE
devops-deployment   2/2     2            2           8h
root@ip-10-0-1-87:~# kubectl get pods -A
NAMESPACE     NAME                                                     READY   STATUS    RESTARTS   AGE
default       devops-deployment-6d7bc59f6c-n74bp                       1/1     Running   0          159m
default       devops-deployment-6d7bc59f6c-tvsmh                       1/1     Running   0          159m
kube-system   metrics-server-746445bd96-d9gsv                          1/1     Running   0          8h
kube-system   metrics-server-746445bd96-j7j4h                          1/1     Running   0          8h
monitoring    alertmanager-kps-kube-prometheus-stack-alertmanager-0    2/2     Running   0          144m
monitoring    alertmanager-monitoring-kube-prometheus-alertmanager-0   2/2     Running   0          6h6m
monitoring    kps-grafana-78d5785fd8-672gr                             3/3     Running   0          144m
monitoring    kps-kube-prometheus-stack-operator-5969cc9b58-fbv9j      1/1     Running   0          144m
monitoring    kps-kube-state-metrics-6696579fb8-9g4np                  1/1     Running   0          144m
monitoring    kps-prometheus-node-exporter-5nxvg                       0/1     Pending   0          144m
monitoring    kps-prometheus-node-exporter-cwbs5                       0/1     Pending   0          144m
monitoring    monitoring-grafana-fc8c66d9-ssqdd                        3/3     Running   0          6h7m
monitoring    monitoring-kube-prometheus-operator-69886c5966-bjrk6     1/1     Running   0          6h7m
monitoring    monitoring-kube-state-metrics-f56d7b58b-86ffp            1/1     Running   0          6h7m
monitoring    monitoring-prometheus-node-exporter-2cwms                1/1     Running   0          6h7m
monitoring    monitoring-prometheus-node-exporter-tzx4t                1/1     Running   0          6h7m
monitoring    prometheus-kps-kube-prometheus-stack-prometheus-0        2/2     Running   0          144m
monitoring    prometheus-monitoring-kube-prometheus-prometheus-0       2/2     Running   0          6h6m
```

Figure 14:Kubernetes Services Exposing the Application

```
aws   Search [Alt+S]   United States (N. Virginia) ▼   Harish (1419-6441-3005) ▼
root@ip-10-0-1-87:~# kubectl get svc -A
NAMESPACE     NAME                                          TYPE           CLUSTER-IP       EXTERNAL-IP   PORT(S)                    AGE
default       devops-service                                LoadBalancer   10.100.172.92    <pending>     80:31181/TCP               8h
default       kubernetes                                    ClusterIP      10.100.0.1       <none>        443/TCP                    9h
kube-system   eks-extension-metrics-api                     ClusterIP      10.100.166.75    <none>        443/TCP                    9h
kube-system   kps-kube-prometheus-stack-coredns             ClusterIP      None             <none>        9153/TCP                   145m
kube-system   kps-kube-prometheus-stack-kube-controller-manager  ClusterIP  None             <none>        10257/TCP                  145m
kube-system   kps-kube-prometheus-stack-kube-etcd           ClusterIP      None             <none>        2381/TCP                   145m
kube-system   kps-kube-prometheus-stack-kube-proxy          ClusterIP      None             <none>        10249/TCP                  145m
kube-system   kps-kube-prometheus-stack-kube-scheduler      ClusterIP      None             <none>        10259/TCP                  145m
kube-system   kps-kube-prometheus-stack-kubelet             ClusterIP      None             <none>        10250/TCP,4194/TCP,10255/TCP  145m
kube-system   metrics-server                                ClusterIP      10.100.23.16     <none>        443/TCP                    8h
kube-system   monitoring-kube-prometheus-coredns            ClusterIP      None             <none>        9153/TCP                   6h7m
kube-system   monitoring-kube-prometheus-kube-controller-manager  ClusterIP  None             <none>        10257/TCP                  6h7m
kube-system   monitoring-kube-prometheus-kube-etcd          ClusterIP      None             <none>        2381/TCP                   6h7m
kube-system   monitoring-kube-prometheus-kube-proxy         ClusterIP      None             <none>        10249/TCP                  6h7m
kube-system   monitoring-kube-prometheus-kube-scheduler     ClusterIP      None             <none>        10259/TCP                  6h7m
kube-system   monitoring-kube-prometheus-kubelet            ClusterIP      None             <none>        10250/TCP,4194/TCP,10255/TCP  6h7m
monitoring    alertmanager-operated                         ClusterIP      None             <none>        9093/TCP,9094/TCP,9094/UDP  6h7m
monitoring    kps-grafana                                   ClusterIP      10.100.63.64     <none>        80/TCP                     145m
monitoring    kps-kube-prometheus-stack-alertmanager        ClusterIP      10.100.151.132   <none>        9093/TCP,8080/TCP          145m
monitoring    kps-kube-prometheus-stack-operator            ClusterIP      10.100.11.146    <none>        443/TCP                    145m
monitoring    kps-kube-prometheus-stack-prometheus          ClusterIP      10.100.119.53    <none>        9090/TCP,8080/TCP          145m
monitoring    kps-kube-state-metrics                        ClusterIP      10.100.90.179    <none>        8080/TCP                   145m
monitoring    kps-prometheus-node-exporter                  ClusterIP      10.100.176.30    <none>        9100/TCP                   145m
monitoring    monitoring-grafana                            ClusterIP      10.100.39.8      <none>        80/TCP                     6h7m
monitoring    monitoring-kube-prometheus-alertmanager       ClusterIP      10.100.201.163   <none>        9093/TCP,8080/TCP          6h7m
monitoring    monitoring-kube-prometheus-operator           ClusterIP      10.100.89.205    <none>        443/TCP                    6h7m
monitoring    monitoring-kube-prometheus-prometheus         ClusterIP      10.100.37.206    <none>        9090/TCP,8080/TCP          6h7m
monitoring    monitoring-kube-state-metrics                 ClusterIP      10.100.152.94    <none>        8080/TCP                   6h7m
monitoring    monitoring-prometheus-node-exporter           ClusterIP      10.100.120.129   <none>        9100/TCP                   6h7m
monitoring    prometheus-operated                           ClusterIP      None             <none>        9090/TCP                   6h7m
root@ip-10-0-1-87:~#
```

4. Continuous Integration (CI)

The CI pipeline automates application build and container image creation.

4.1 CI Pipeline Implementation (GitHub Actions)

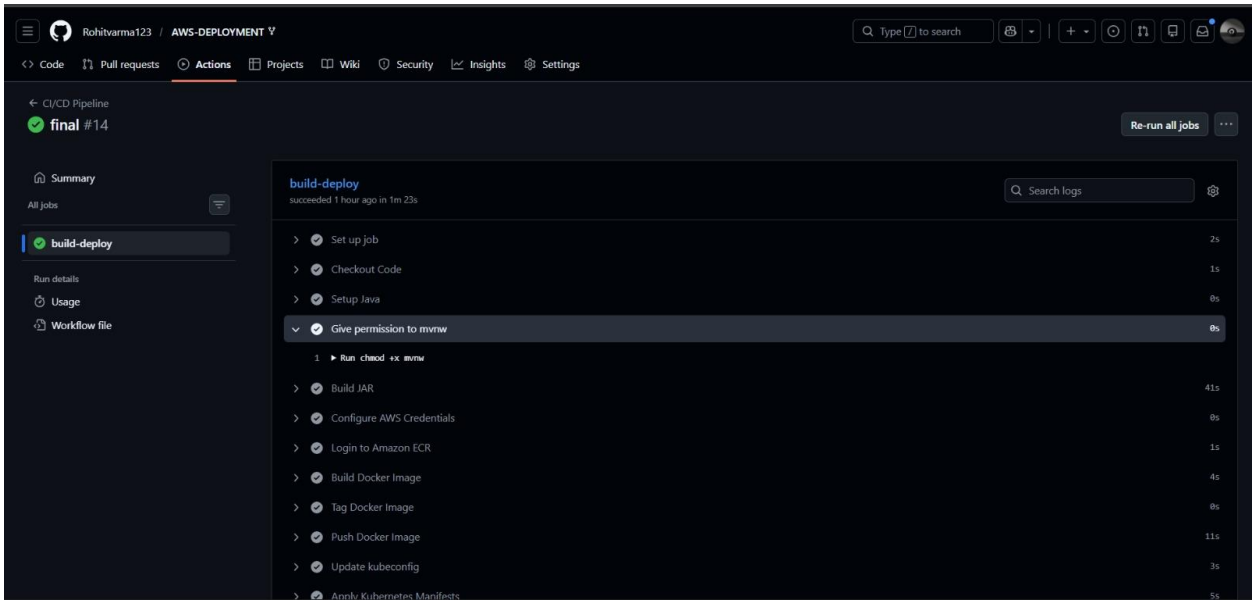
The CI pipeline is implemented using **GitHub Actions** to automate build, scanning, and image creation. name: CI/CD Pipeline

```
on:
push:
branches:
- main
env:
AWS_REGION: us-east-1
ECR_REPOSITORY: devops-ecr
EKS_CLUSTER_NAME: devops
IMAGE_TAG: ${{ github.sha }}

jobs:
build-deploy:
runs-on: ubuntu-latest
steps:
- name: Checkout Code
uses: actions/checkout@v3
- name: Setup Java
uses: actions/setup-java@v3
with:
distribution: 'temurin'
```

```
      java-version: '11'

    - name: Give permission to mvnw
      run: chmod +x mvnw

    - name: Build JAR
      run: ./mvnw clean package -DskipTests

    - name: Configure AWS Credentials
      uses: aws-actions/configure-aws-credentials@v2
      with:
        aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
        aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
        aws-region: ${{ env.AWS_REGION }}

    - name: Login to Amazon ECR
      uses: aws-actions/amazon-ecr-login@v1

    - name: Build Docker Image
      run: docker build -t $ECR_REPOSITORY:$IMAGE_TAG .

    - name: Tag Docker Image
      run: docker tag $ECR_REPOSITORY:$IMAGE_TAG ${{ secrets.AWS_ACCOUNT_ID
}}.dkr.ecr.$AWS_REGION.amazonaws.com/$ECR_REPOSITORY:$IMAGE_TAG
```

```
    - name: Push Docker Image

      run: docker push ${{ secrets.AWS_ACCOUNT_ID
}}.dkr.ecr.$AWS_REGION.amazonaws.com/$ECR_REPOSITORY:$IMAGE_TAG



    - name: Update kubeconfig

      run: aws eks update-kubeconfig --region $AWS_REGION --name
$EKS_CLUSTER_NAME



    - name: Apply Kubernetes Manifests

      run: |

        kubectl apply -f k8s/deployment.yaml

        kubectl apply -f k8s/service.yaml



    - name: Deploy to EKS

      run: |

        kubectl set image deployment/devops-deployment devops-container=${{
secrets.AWS_ACCOUNT_ID
}}.dkr.ecr.$AWS_REGION.amazonaws.com/$ECR_REPOSITORY:$IMAGE_TAG

        kubectl rollout status deployment/devops-deployment



    - name: Get LoadBalancer Endpoint

      run: |

        echo "LoadBalancer Endpoint:"

        kubectl get svc devops-service -o jsonpath='{.status.loadBalancer.ingress[0].hostname}'
```

Figure 15: CI Pipeline Build Stage Completed



CI Workflow Steps

1. **Source Code Checkout**

The pipeline fetches the latest code from the Git repository.

---

2. **Build Application Using Maven**

The Java application is compiled and packaged using Maven.

Command example:

```
mvn clean package
```

---

3. **Code Quality Analysis**

The code is scanned using SonarQube to detect bugs, vulnerabilities, and code smells.

Example command:

```
mvn sonar:sonar
```

---

4. **Docker Image Build**

A Docker image is built from the application source code.

Example command:

```
docker build -t application-image .
```

---

5. **Security Scan Using Trivy**

The Docker image is scanned for vulnerabilities using Trivy.

Example command:

```
trivy image application-image
```

---

6. **Push Image to Amazon ECR**

After successful scanning, the image is pushed to the container registry.

Steps:

- Authenticate Docker with ECR
- Tag the image
- Push image

5. Continuous Deployment (CD)

After the image is pushed to the registry, the CD pipeline deploys the application to Kubernetes.

Deployment can be performed using:

- Kubernetes manifest files
- Helm charts

Deployment Steps

1. Pull container image from ECR.
2. Deploy application to Kubernetes cluster.

Example Kubernetes deployment command:

```
kubectl apply -f deployment.yaml
```

Helm Deployment Example

```
helm install application-chart ./helm-chart
```

6. CI/CD Pipeline Flow

Developer Push Code

│

▼

CI Pipeline Triggered

│

▼

Build Application (Maven)

│

▼

Code Quality Check (SonarQube)

│

▼

Build Docker Image

│

▼

Security Scan (Trivy)

│

▼

Push Image to ECR

│

▼

CD Pipeline Triggered

|

▼

Deploy to Kubernetes Cluster

|

▼

Application Running on EKS/EC2

|

▼

Monitoring Prometheus And Grafana

7. Deployment Verification

After deployment, the application can be verified using:

- Kubernetes pods status

```
kubectl get pods
```

- Services

```
kubectl get svc
```

- Application endpoint

```
http://<LoadBalancer-IP>
```

---

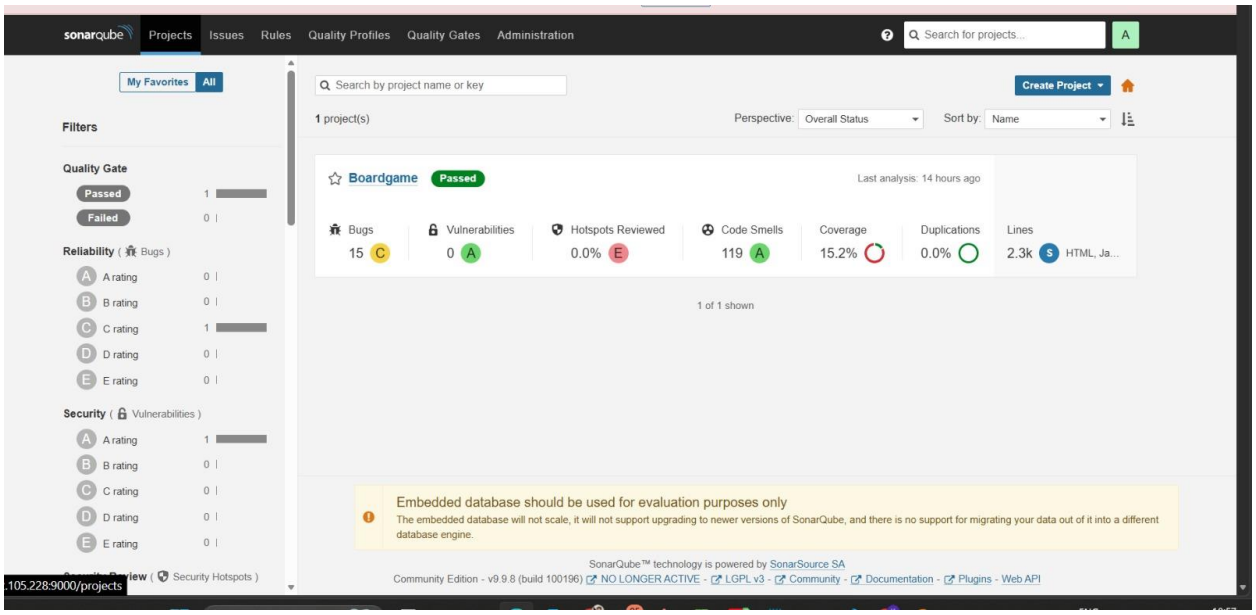
8. Monitoring and Observability

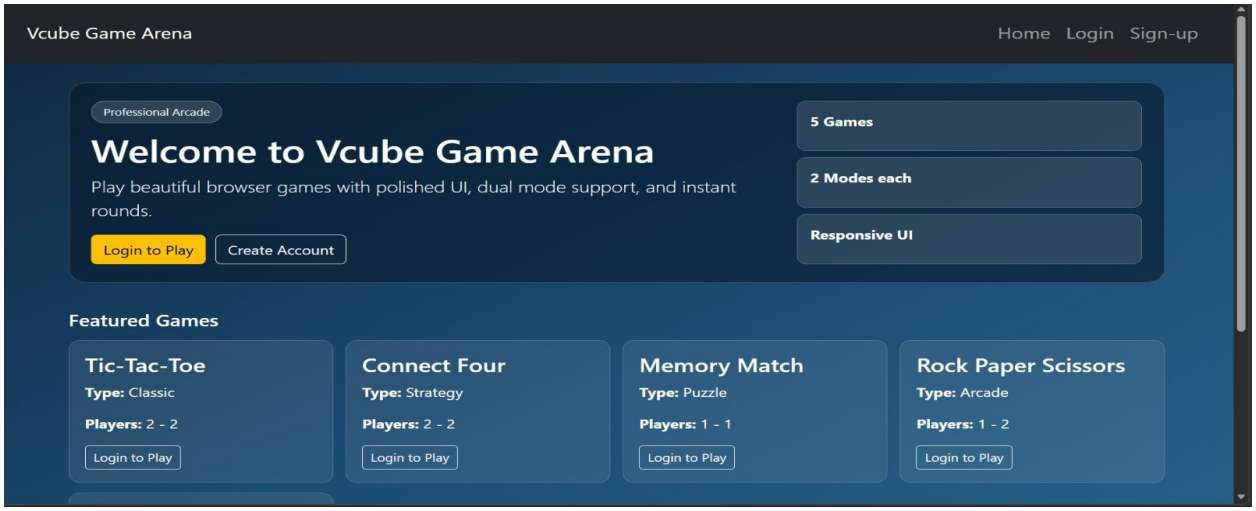
Monitoring tools installed on the DevOps server provide system metrics and visualization.

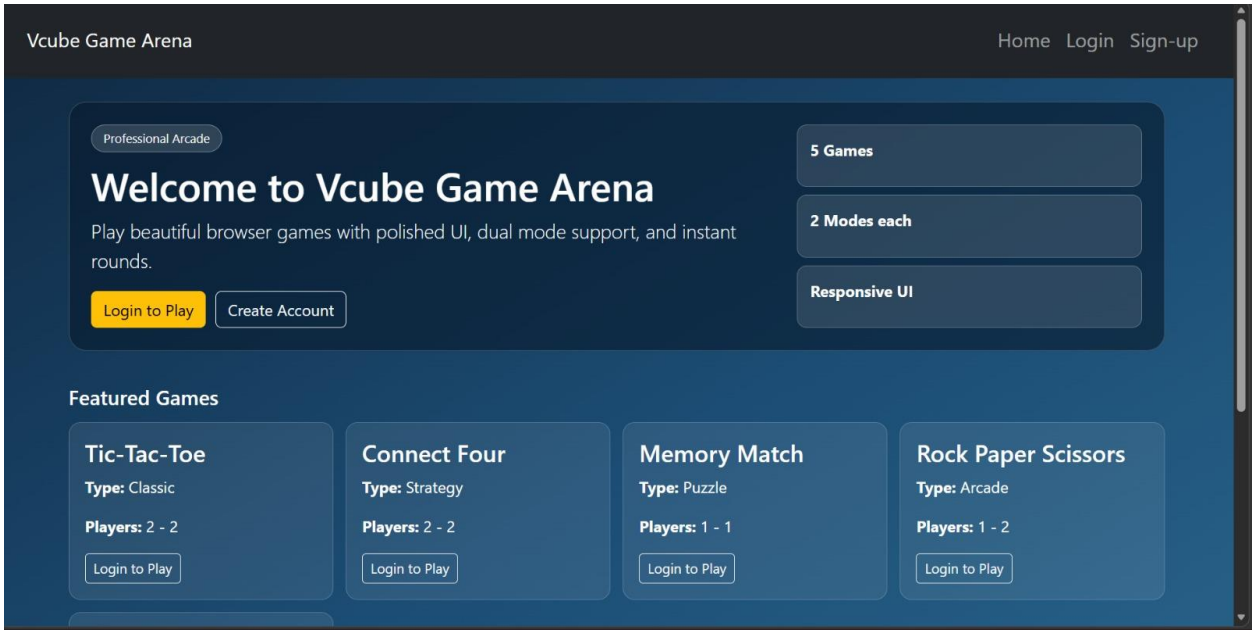
Monitoring stack:

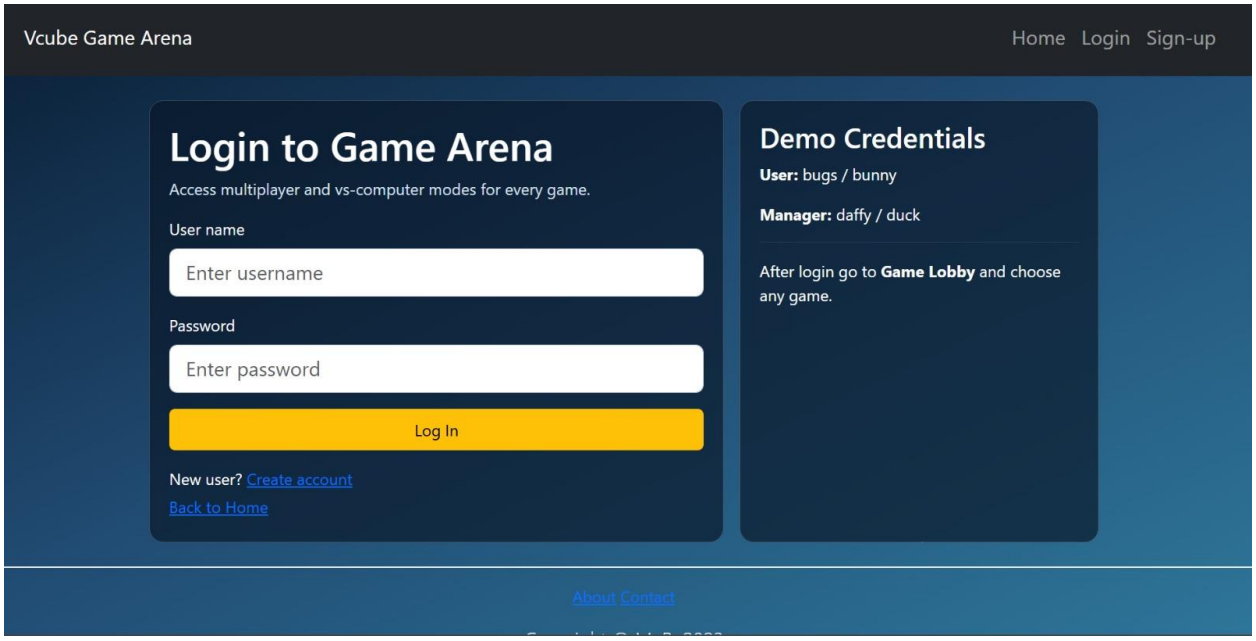
- Prometheus – metrics collection
- Grafana – dashboard visualization
- Node Exporter – system metrics

Outputs




Vcube Game Arena
Home Login Sign-up
Professional Arcade
Welcome to Vcube Game Arena
Play beautiful browser games with polished UI, dual mode support, and instant rounds.
Login to Play
Create Account
5 Games
2 Modes each
Responsive UI
Featured Games
Tic-Tac-Toe
Type: Classic
Players: 2 - 2
Login to Play
Connect Four
Type: Strategy
Players: 2 - 2
Login to Play
Memory Match
Type: Puzzle
Players: 1 - 1
Login to Play
Rock Paper Scissors
Type: Arcade
Players: 1 - 2
Login to Play


Vcube Game Arena
Home Login Sign-up
Professional Arcade
Welcome to Vcube Game Arena
Play beautiful browser games with polished UI, dual mode support, and instant rounds.
Login to Play
Create Account
5 Games
2 Modes each
Responsive UI
Featured Games
Tic-Tac-Toe
Type: Classic
Players: 2 - 2
Login to Play
Connect Four
Type: Strategy
Players: 2 - 2
Login to Play
Memory Match
Type: Puzzle
Players: 1 - 1
Login to Play
Rock Paper Scissors
Type: Arcade
Players: 1 - 2
Login to Play



9. Benefits of the CI/CD Pipeline

- Automated build and deployment
- Faster release cycles
- Improved code quality
- Continuous security scanning
- Scalable containerized deployments

---

10. Conclusion

The CI/CD pipeline successfully automates the entire software delivery lifecycle from code commit to application deployment.

By integrating Terraform infrastructure provisioning with CI/CD automation, the system ensures consistent, reliable, and scalable application deployments on AWS.